CHRIST
(DEEMED TO BE UNIVERSITY)
BANGALORE | DELHI NCR | PUNE

HELIX

A Multi-Tenant Collaborative AI Workspace with Branchable Conversations and a Monitored Deep-Reasoning Mode

by

Achindra Sharma (2547105)
Rajnish Kumar (2547143)
M M Mohamed Mansoor (2547132)

Under the guidance of

<Guide Name>

Specialization Project report submitted in partial fulfillment of
the requirements of IV trimester MCA,

CHRIST (Deemed to be University)

August 2026

CERTIFICATE

This is to certify that the report titled "Helix — A Multi-Tenant Collaborative AI Workspace with Branchable Conversations and a Monitored Deep-Reasoning Mode" is a bona fide record of work done by Achindra Sharma (2547105), Rajnish Kumar (2547143) and M M Mohamed Mansoor (2547132), of CHRIST (Deemed to be University), Bangalore, in partial fulfillment of the requirements of IV Trimester MCA during the year 2026.

Project Guide **Head of the Department**

Valued by:

Name :

Register Number :

Examination Centre : CHRIST (Deemed to be University)

Date of Exam :

ACKNOWLEDGEMENTS

We express our sincere gratitude to our project guide for the continued guidance and encouragement throughout the development of Helix. We thank the Department of Computer Science, CHRIST (Deemed to be University), for providing the resources and the academic environment that made this work possible, and our families and peers for their constant support.

ABSTRACT

Teams increasingly use AI for knowledge work, but today's AI tools are built for one person in one private tab. As a result, teammates run prompts in isolation, cannot see or continue each other's conversations, and keep no record of what has already been tried — causing duplicated effort, wasted API budget, and lost progress.

Helix is a multi-tenant collaborative AI workspace that addresses this gap. A team shares one workspace where conversations can be shared or private, any thread can be forked and branched to explore alternatives, and a shared prompt library captures what works. Live updates and presence make the workspace feel shared. For hard, open-ended problems, any conversation can be escalated into a Deep Reasoning mode — a recursive reasoning run the whole team can watch live, with a kill switch, a steer/pause control, and a cost-budget meter so a long autonomous run never becomes an opaque, runaway black box. Helix is model-agnostic, running on either hosted (Groq) or local (Ollama) LLMs, so teams can adopt it on their own terms. The project demonstrates three core technical concepts — Agentic AI (the recursive engine), Distributed Systems (real-time multi-client synchronisation), and Security/RBAC (strict multi-tenancy with a per-action permission policy).

TABLE OF CONTENTS

LIST OF FIGURES

LIST OF TABLES

CHAPTER 1: INTRODUCTION

This chapter introduces the project, the motivation and problem it addresses, its objectives, related existing works, the expected benefits, the project's purpose, scope and applicability, and the plan of work.

1.1 Background and Motivation

AI assistants such as ChatGPT, Claude, Gemini, and locally run open models have become a routine part of how engineering teams, student project groups, and research teams do knowledge work — but the tools have not caught up to the fact that this work is collaborative. Knowledge gained in one person's session is invisible to everyone else; the same prompts get re-written from scratch; and when a teammate is away, nobody can pick up their thread. As teams begin using AI for longer, more autonomous reasoning, a second gap appears: those runs are opaque and can quietly burn time and budget.

The motivation for Helix is to make team AI work shared, reusable, and — when it gets ambitious — observable and controllable. By capping and controlling long autonomous runs and eliminating duplicated compute, the project also aligns with Sustainable Development Goal 12 (Responsible Consumption and Production).

1.2 Problem Statement

When teams do knowledge work with AI together, three problems follow:

1. Siloed context — Teammates cannot see each other's conversations or reuse what has already been tried, causing redundant queries, wasted budget, and lost progress when someone needs to pick up where a colleague left off.
2. No branching — Linear chats cannot fork, so a promising thread cannot be cloned to explore an alternative without disrupting the original.
3. Unmonitored autonomy — Long, autonomous AI runs are opaque: no mid-run visibility, no stop control, and no cost guardrail — risking wasted compute and runaway cost before anyone can intervene.

1.3 Objectives

1. Provide multi-tenant workspaces with accounts, invite-link onboarding, and roles (Owner / Collaborator / Observer), with strict per-tenant isolation.
2. Support shared and private conversations within a workspace.

3. Deliver real-time sync and presence so members see updates and streaming responses live.
4. Implement fork and branch of any conversation as a persistent tree.
5. Build a shared prompt library with tags and search to capture reusable prompts.
6. Provide a Deep Reasoning mode: an escalation into a recursive reasoning run.
7. Provide a monitor for Deep Reasoning runs — live trace, kill switch, steer/pause, and a token/cost budget meter.
8. Persist conversations and runs for replay and export (JSON / Markdown).
9. Remain model-agnostic and cost-aware — runnable on hosted or local LLMs, with explicit controls on long-run token spend.

1.4 Existing Works

Each existing category solves only one slice of collaborative AI work; Helix's contribution is the combination of shared and branchable conversations, a reusable prompt library, and a monitored deep-reasoning mode in one workspace.

Table 1.1: Existing works and the gap Helix fills

System	Strength	Gap Helix fills
ChatGPT / Claude (web)	Strong single-user chat	One person, one tab — no sharing, branching, or library
ChatGPT Team / Claude Projects	Shared files & history	Asynchronous sharing only — no live branchable conversations or reusable prompt library
Prompt managers (e.g. PromptHub)	Prompt storage	Standalone — not tied to live team conversations or reasoning
Agent frameworks (LangGraph, etc.)	Agent orchestration	Single-developer; no collaboration, no monitoring/kill switch out of the box

1.5 Benefits of the Proposed Work

- No duplicated effort — shared conversations and a prompt library mean the team reuses what works instead of re-asking.
- Continuity — anyone can pick up where a teammate left off.
- Parallel exploration — branching lets the team try competing approaches without losing context.

- Cost and oversight — the monitor caps and controls long autonomous runs, cutting wasted compute (SDG 12).
- Flexible deployment — model-agnostic, running on hosted or local LLMs, so teams choose by cost, privacy, or latency.

1.6 Purpose, Scope and Applicability

1.6.1 Purpose

The purpose of Helix is to remove the black boxes from team AI work — to make a team's reasoning shared, branchable, reusable and, for autonomous runs, observable and controllable. It improves the current way of working by replacing isolated private tabs with one shared, recorded workspace, and by replacing opaque autonomous runs with a governed, watchable one.

1.6.2 Scope

The project delivers a working multi-tenant web application: authentication and workspaces, shared/private conversations with real-time sync, fork-and-branch, a shared prompt library, a Deep Reasoning mode with a live monitor (kill / steer / budget), and history replay and export. It assumes a small team per workspace and a single backend instance for the core demonstration (horizontal scaling via Redis pub/sub is an optional extension). Inference runs through a pluggable provider on either hosted (Groq) or local (Ollama) models.

1.6.3 Applicability

Helix is directly applicable to software engineering teams, student project groups, and research teams that use AI collaboratively. Indirectly, by capping and controlling autonomous runs it contributes to responsible, cost-aware use of compute, and by being model-agnostic it supports privacy-sensitive deployments on fully local models.

1.7 Plan of Work

The work spans approximately twelve weeks (2 June 2026 – 25 August 2026) across a team of three. The collaborative core is built first on plain chat, after which the Deep Reasoning mode and its monitor are added. Responsibility is split as: Achindra Sharma — Deep Reasoning engine and integration; Rajnish Kumar — backend and infrastructure; M M Mohamed Mansoor — frontend and UX (roles may be adjusted by mutual agreement).

Table 1.2: Plan of Work

Task / Module	Person Responsible	Duration
Project setup — monorepo, Docker Compose (Postgres / Redis / Ollama), shared schemas	All	02 Jun – 08 Jun

Task / Module	Person Responsible	Duration
M1 — Auth & Workspace (accounts, JWT, workspaces, invite links, roles, DB schema)	Rajnish Kumar	09 Jun – 22 Jun
M2 — Conversations (provider interface Groq/Ollama; shared & private conversations; streaming chat)	Achindra Sharma	09 Jun – 22 Jun
Auth & workspace UI (login/register, workspace dashboard, chat view)	M M Mohamed Mansoor	09 Jun – 22 Jun
M3 — Real-Time Sync & Presence (WebSocket rooms, presence, ordered message log)	Rajnish Kumar	23 Jun – 06 Jul
Shared context (append to shared thread, token budgeting, summarisation)	Achindra Sharma	23 Jun – 06 Jul
Collaboration UI (multi-user shared view, presence avatars, stream fan-out)	M M Mohamed Mansoor	23 Jun – 06 Jul
M4 — Fork & Branch Tree (fork via parent pointers; branch context)	Achindra Sharma / Rajnish Kumar	07 Jul – 13 Jul
M5 — Shared Prompt Library (save / tag / search prompts; reuse)	M M Mohamed Mansoor	14 Jul – 20 Jul
M6 — Deep Reasoning Mode (recursive engine; escalation path; checkpoint fork)	Achindra Sharma	21 Jul – 03 Aug
M7 — Monitor (backend): persist runs/steps, stream events, kill/steer endpoints, budget metering	Rajnish Kumar	21 Jul – 10 Aug
M7 — Monitor (UI): Deep Reasoning button, live trace/topology, kill switch, steer, budget meter	M M Mohamed Mansoor	28 Jul – 10 Aug
M8 — History, Replay & Export (replay any branch; JSON/Markdown export)	Achindra Sharma	11 Aug – 17 Aug
M9 — Permission Layer (role-gated actions; tool allowlist for Deep Reasoning)	Rajnish Kumar	11 Aug – 17 Aug
Hardening & demo (tenant isolation/RLS, deployment, performance test, demo video, README)	All	18 Aug – 25 Aug

1.8 Overview of the Report

The remainder of this report is organised as follows. Chapter 2 — System Analysis and Requirements — studies the existing systems and their limitations, defines the proposed system, its benefits and features, and specifies the functional, non-functional, software and hardware requirements. Chapter 3 — System Design — presents the system architecture, module design, data-flow and ER diagrams, database design, user-interface design, and the algorithm design for the core mechanisms of Helix.

CHAPTER 2: SYSTEM ANALYSIS AND REQUIREMENTS

This chapter studies the existing systems and their limitations, defines the proposed system with its benefits and features, and specifies the system requirements.

2.1 Existing System

AI assistants such as ChatGPT, Claude, Gemini, and locally run open models have become a routine part of team knowledge work, yet almost every mainstream tool is built around a single assumption: one user, one private conversation. Studying how teams actually use these tools reveals five distinct categories of existing system, each solving only one slice of collaborative AI work.

Table 2.1: Categories of existing systems

#	Category	Representative tools	What it does well	The slice it owns
1	Single-user chat assistants	ChatGPT, Claude.ai, Gemini	Fast, high-quality single-user question answering	The conversation itself
2	Team AI suites	ChatGPT Team, Claude Projects / Teams	Shared files, shared history, seat & billing management	Sharing artifacts and history
3	Prompt managers	PromptLayer, ad-hoc notebooks / spreadsheets	Storing and organising prompts	The record of prompts
4	Agent frameworks	LangChain, LangGraph, AutoGPT, CrewAI	Orchestrating autonomous multi-step agents	The reasoning loop
5	Agent observability	LangSmith, Langfuse, Helicone	Tracing & monitoring agent runs after they finish	Visibility into runs

When a team faces a hard, open-ended problem, the work fragments without anyone deciding it should: members open parallel private tabs that none of the others can see; useful fragments are copy-pasted into a chat channel stripped of their surrounding context; trying a different angle means overwriting or restarting a thread and losing the original line of thought; a carefully crafted prompt that worked last week lives in one person's history and is rewritten from scratch by the next person; and for genuinely hard questions a member might wire up an agent framework locally whose run is invisible and uncontrollable to the rest of the team.

No single existing system combines all four of the capabilities a collaborating team needs at once: a shared, live conversation surface; branchable threads that preserve lineage; a reusable, searchable prompt record; and a governed, watchable autonomous reasoning mode. Helix exists to close those seams in one product.

2.2 Limitations of the Existing System

The study above translates into seven concrete limitations that the proposed system must overcome.

- L1 — Isolation and siloed context. Conversations are private by default; a colleague cannot see, continue, or learn from another member's thread, so the team repeatedly re-asks questions already answered.
- L2 — No branching; lossy exploration. Mainstream chat tools present a single linear thread, and editing an earlier message destroys the alternative rather than preserving it.
- L3 — No shared, reusable prompt record. Working prompts are trapped in one person's history or informal documents; there is no workspace-scoped, tagged, searchable library.
- L4 — Autonomy without governance. Where agent frameworks offer multi-step autonomous reasoning, the run is a black box: no live trace, no kill switch, no mid-flight steer, and no cost ceiling.
- L5 — Weak team and tenancy model. Single-user tools have no workspace roles; team suites add seats but not fine-grained, per-action permissions.
- L6 — Observability is post-hoc and separate. Observability tools trace runs after they complete, inside a different product, with no in-the-moment control.
- L7 — Lock-in and cost. Most team AI is billed per seat or per token against a single hosted provider, with no model-agnostic local option.

2.3 Proposed System

The core problem is that collaborative AI work is forced through single-user tools, so context is siloed, exploration is lossy, prompts are not reused, and autonomous runs are ungoverned. Helix decomposes this into sub-problems — tenancy and access (L5), shared and private conversations (L1), branching (L2), prompt reuse (L3), real-time collaboration (L1, L6), governed autonomous reasoning (L4, L6), and model flexibility (L7) — and solves each as a feature of one product.

Helix is a multi-tenant collaborative AI workspace — informally, "Git for your team's AI work." Instead of everyone prompting alone in private tabs, a team shares one workspace where AI conversations are shared, branchable, recorded, and — when a problem is hard enough — escalated into a monitored deep-reasoning run.

2.4 Benefits of the Proposed System

- Eliminates duplicated effort through shared conversations and a reusable prompt library.
- Provides continuity — any member can resume a teammate's thread.

- Enables parallel exploration via O(1) forking that preserves lineage.
- Governs cost and autonomy through a live monitor with kill switch, steer/pause and a budget meter (SDG 12).
- Offers flexible, model-agnostic deployment on hosted (Groq) or local (Ollama) models.

2.5 Features of the Proposed System

Each feature directly answers one or more of the limitations identified in Section 2.2.

Table 2.2: Features of the proposed system

Feature	Answers	Description
F1 — Multi-tenant workspaces with RBAC	L5	Authenticated accounts, invite-link onboarding, three roles (Owner/Collaborator/Observer); every resource scoped to one isolated workspace tenant.
F2 — Shared and private conversations	L1	A conversation belongs to a workspace and is shared or private; AI responses stream token-by-token. Shared conversations become durable team knowledge.
F3 — Fork and branch tree	L2	Any conversation can be forked into an independent branch that inherits parent context but evolves separately, visualised as a persistent Git-style tree. Forking copies no history.
F4 — Shared prompt library	L3	Working prompts are saved with tags and full-text search, and can be inserted into any conversation for reuse.
F5 — Real-time collaboration and presence	L1, L6	A live channel per workspace broadcasts new messages, streaming tokens, and presence signals.
F6 — Deep Reasoning mode	L4	A hard question can be escalated into a recursive engine that reasons, reflects on its own thinking, and synthesises — looping until it converges.
F7 — Monitor and run control	L4, L6	A live monitor with a reasoning trace and topology view, a kill switch, a steer/pause control, a recursion-depth guard, and a token-cost budget meter with alerts.
F8 — Model-agnostic provider layer	L7	All inference flows through one provider interface with interchangeable hosted (Groq) and local (Ollama) backends, selected by configuration.
F9 — History, replay, and export	L1, L6	Conversations and runs are persisted; any branch can be replayed step by step and exported as JSON or Markdown.

Together these features demonstrate the project's three core technical concepts: Agentic AI (the recursive Deep Reasoning engine), Distributed Systems (real-time multi-client synchronisation, ordering, and fan-out), and Security / RBAC (strict multi-tenancy with a per-action permission policy).

2.6 System Requirements Specification

This section defines the requirements of the system without focusing on how they will be achieved.

2.6.1 User Characteristics

Helix serves three kinds of user, distinguished by role within a workspace:

- Owner — creates the workspace, invites and manages members and roles, edits the permission policy, and has full authority over conversations and runs.
- Collaborator — a working member who can send messages, fork conversations, save and use library prompts, escalate to Deep Reasoning, and steer or kill runs.
- Observer — a read-only member who can view, replay and export conversations and watch Deep Reasoning runs, but cannot act on them.

2.6.2 Software and Hardware Requirements

(i) Software Requirements

Table 2.3: Software requirements

Category	Requirement
Operating System	Cross-platform; Linux containers for deployment
Frontend	React, TypeScript, TailwindCSS, Vite; modern browser
Backend	Python 3.11+, FastAPI, WebSockets
Deep Reasoning engine	LangGraph (recursive reasoning, checkpointing)
LLM inference	Pluggable provider — Groq (hosted) or Ollama (local), via an OpenAI-compatible interface
Database	PostgreSQL (production) / SQLite (development)
Containerisation	Docker + Docker Compose (local development)
Version control	Git / GitHub
Optional (scaling only)	Redis — pub/sub fan-out across multiple backend instances

(ii) Hardware Requirements

Table 2.4: Hardware requirements

Component	Minimum (development)	Notes
CPU	Quad-core x86-64	Backend, DB, containers

Component	Minimum (development)	Notes
RAM	8 GB (16 GB recommended)	~8 GB if running local Ollama; otherwise use Groq
Storage	10 GB free	Docker images, DB, optional model weights
Network	Broadband	Required for hosted inference (or run Ollama locally)
Client	Any modern desktop/laptop with a browser	No special hardware
Deployment	Any container host	No owned servers required

2.6.3 Constraints

- Hardware limitations — running a local Ollama model requires roughly 8 GB of RAM; otherwise hosted Groq inference is used.
- Language / platform — Python 3.11+ on the backend, TypeScript/React on the frontend, and a modern browser on the client.
- Interfaces to other applications — LLM access via an OpenAI-compatible interface (Groq / Ollama); persistence via PostgreSQL/SQLite; optional Redis for pub/sub.
- Reliability — every Deep Reasoning run must be haltable at the next step by the kill switch, bounded by recursion-depth, loop and compute-budget guards.
- Regulatory / privacy — only the data needed to operate is collected; no biometric, audio or video data is captured.

2.6.4 Functional Requirements

Table 2.5: Functional Requirements

ID	Requirement	Description
FR-1	Authentication & Accounts	Registration and login with hashed credentials; issues signed JWT tokens for REST and real-time endpoints.
FR-2	Workspaces & Multi-Tenancy	Create workspaces, onboard members via invite links, and scope every resource to one isolated workspace tenant.
FR-3	Role-Based Access Control	Assign each member a role (Owner/Collaborator/Observer) and authorise every action against a role-to-permission policy table.
FR-4	Shared & Private Conversations	Create shared or private conversations and stream AI responses token-by-token.

ID	Requirement	Description
FR-5	Real-Time Sync & Presence	Maintain one real-time room per workspace broadcasting messages, tokens and presence, with a single ordered append-only log.
FR-6	Conversation Fork & Branch Tree	Fork any conversation at any point into a child branch that inherits context and evolves independently, persisted as a branch tree.
FR-7	Shared Prompt Library	Save prompts with tags to a workspace library, search/browse it, and insert a saved prompt into a conversation.
FR-8	LLM Provider Abstraction	Route all inference through one provider interface with interchangeable Groq/Ollama backends, selectable by configuration.
FR-9	Deep Reasoning Mode	Escalate a conversation into a recursive reason-reflect-synthesize run emitting a structured step event per transition.
FR-10	Deep Reasoning Monitor	A live dashboard consuming step/token events, showing the trace, topology, and energy/depth/loop-guard values in real time.
FR-11	Run Control (Kill & Steer)	An authorised kill switch halting at the next step boundary, and a steer control that pauses and resumes a run with injected guidance.
FR-12	Budget Meter & Guardrails	Meter token usage and request rate per workspace against thresholds with alerts, and bound run spend via a compute-budget halt.
FR-13	History, Replay & Export	Persist conversations and runs, replay any branch step by step, and export as JSON or Markdown.
FR-14	Permission Layer	Owner-restricted tool allowlist for Deep Reasoning and human approval before high-risk tool execution, layered on RBAC.

2.6.5 Non-Functional Requirements

Table 2.6: Non-Functional Requirements

ID	Attribute	Description
NFR-1	Performance & Latency	Real-time events (message broadcast, presence, token fan-out) reach clients with a target latency under 200 ms.
NFR-2	Multi-Tenancy & Isolation	Tenant isolation via workspace-scoped queries and database row-level security; no data crosses tenant boundaries.
NFR-3	Cost Efficiency	Long-run token spend bounded by a compute-budget halt and per-workspace meter; runs on free hosted (Groq) or local (Ollama) models.
NFR-4	Scalability	Single-instance in-memory rooms; optional pub/sub layer for horizontal scaling without session affinity.

ID	Attribute	Description
NFR-5	Security	All endpoints protected by token auth and RBAC; Deep Reasoning tool use limited to a safe allowlist.
NFR-6	Reliability & Interruptibility	Any run is haltable at the next step; recursion-depth and compute-budget guards prevent runaway loops; safe error reporting.
NFR-7	Streaming & Backpressure	A single response stream fans out to many clients without a slow client stalling the others.
NFR-8	Privacy	Only operational data is collected; no biometric, audio, or video data is captured.
NFR-9	Portability	Fully containerisable; the provider interface allows new LLM backends without application changes.

2.7 Block Diagram

The block diagram below models the system under study — the React client, the FastAPI backend (auth/RBAC gate, services, provider layer, sync layer, Deep Reasoning engine) and the relational database — and the major flows between them (REST/SSE for authoritative reads/writes, WebSocket for presence and broadcast, and the pluggable LLM backend).

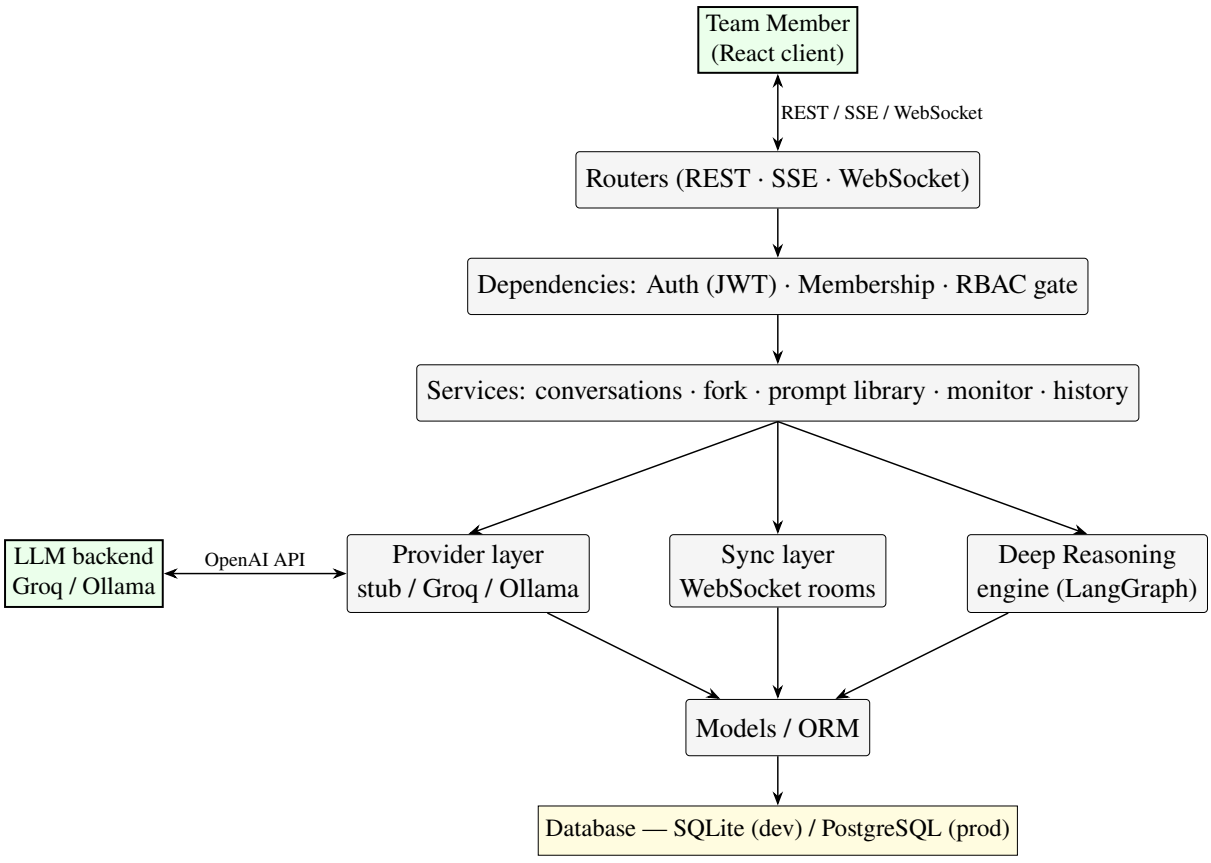

Figure 2.1: System block / context diagram of Helix

CHAPTER 3: SYSTEM DESIGN

This chapter describes the architecture, modules, data and interface design of Helix, and the algorithm design for its core mechanisms.

3.1 System Architecture

Helix is a three-tier application — a React client, a single FastAPI backend, and a relational database — organised so that each kind of traffic uses the transport that fits it, and each backend concern lives in its own module with one clear job. The transport split is deliberate.

Table 3.1: Transport split across REST, SSE and WebSocket

Transport	Carries	Why this transport
REST (JSON)	All authoritative reads/writes: auth, workspaces, conversations, forks, prompts, run control	One ordered, idempotent, cacheable surface for state changes
SSE (Server-Sent Events)	The LLM token stream back to the sender	One-way, long-lived, simple — exactly a token stream
WebSocket (one room per workspace)	Presence and read-only broadcast of what already happened to other members	Bidirectional liveness used as fan-out, so the server keeps authoritative order

Sends and forks go through REST/SSE, where the server stamps a monotonic order, while the WebSocket room is a pure broadcast channel. This avoids operational-transform/CRDT complexity while keeping a single, consistent shared log. Requests flow inward through clearly separated layers: routers (transport, validation), dependencies (auth, membership, RBAC gate), services (conversations, fork, library, monitor, history), the provider / sync / engine layer, and finally models / ORM and the database.

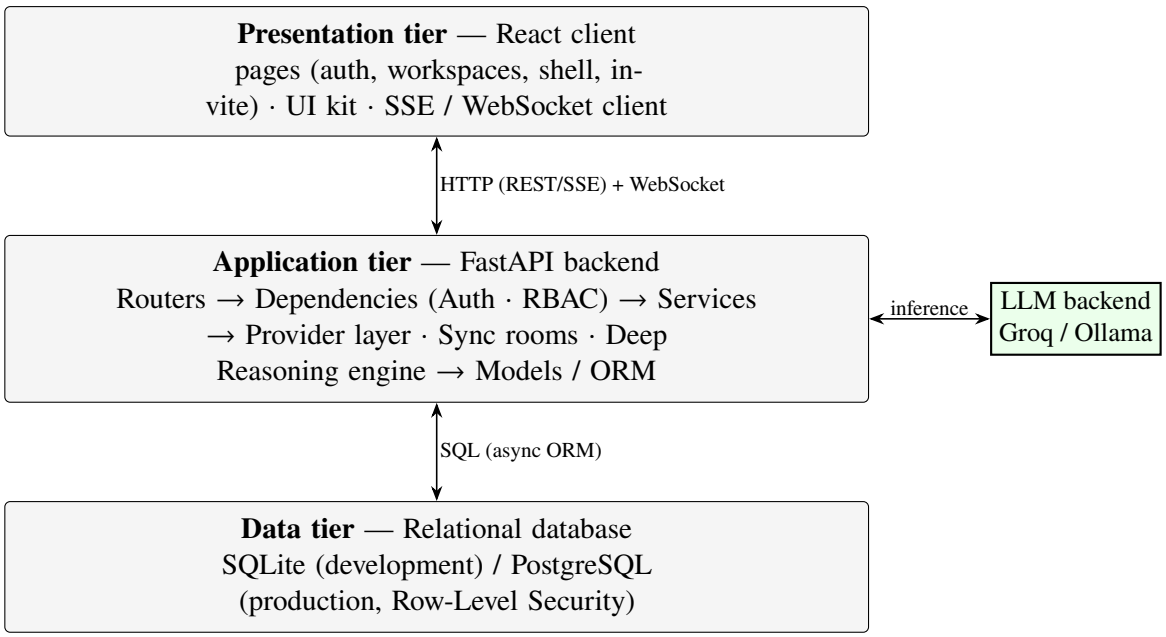

Figure 3.1: Three-tier architecture and backend layering

3.2 Module Design

Following a divide-and-conquer approach, the backend is decomposed into modules, each mapping to a concrete place in the codebase with one clear responsibility. The frontend modules mirror the surfaces: an API client and auth context, route-level page components (auth, workspaces, workspace shell, invite), and a shared UI-component kit.

Table 3.2: Module decomposition

Module	Responsibility	Location
Config	Typed settings from env/.env (DB URL, JWT, provider, URLs)	backend/api/config.py
DB / Session	Async engine, session-per-request, schema bootstrap, health ping	backend/api/db.py
Models	ORM entities + small domain helpers (role rank, expiry, UTC normalise)	backend/api/models.py
Security	bcrypt hashing, JWT encode/decode	backend/api/security.py
Deps (Auth + RBAC)	get_current_user, get_membership, require_role gate	backend/api/deps.py
Auth router	Register, login, /me	routers/auth.py
Workspaces router	Workspaces, invites, members, role patch	routers/workspaces.py
Provider layer	One streaming LLMProvider interface; stub/Groq/Ollama backends	backend/api/providers/*
Conversation service	Create conversations, persist nodes, drive the SSE send	routers/conversations.py
Fork service	O(1) fork; resolve history across branch boundaries; branch tree	services/fork.py

Module	Responsibility	Location
Sync / WS rooms	One in-memory room per workspace; presence; broadcast; per-client queues	ws/room.py
Prompt library	Save/search/tag prompts; usage counter	routers/prompts.py
Deep Reasoning engine	Recursive reason-reflect-synthesize loop; checkpointing; step/token events	backend/engine/*
Monitor service	Normalise engine events; meter tokens/rate vs thresholds	services/monitor.py
Run controller	Start/steer/kill; depth & loop guards; RBAC-gated	routers/runs.py
History service	Replay a branch; export JSON/Markdown	routers/history.py
Permission policy	Owner-editable RBAC matrix; tool allowlist + approval	routers/permissions.py

Dependencies point inward and downward; nothing in a lower layer imports a router. The two relationships that look bidirectional — Conversation/Sync and Engine/Monitor — are event-based (a service emits to the sync layer; it does not call back into the router), so there is no import cycle.

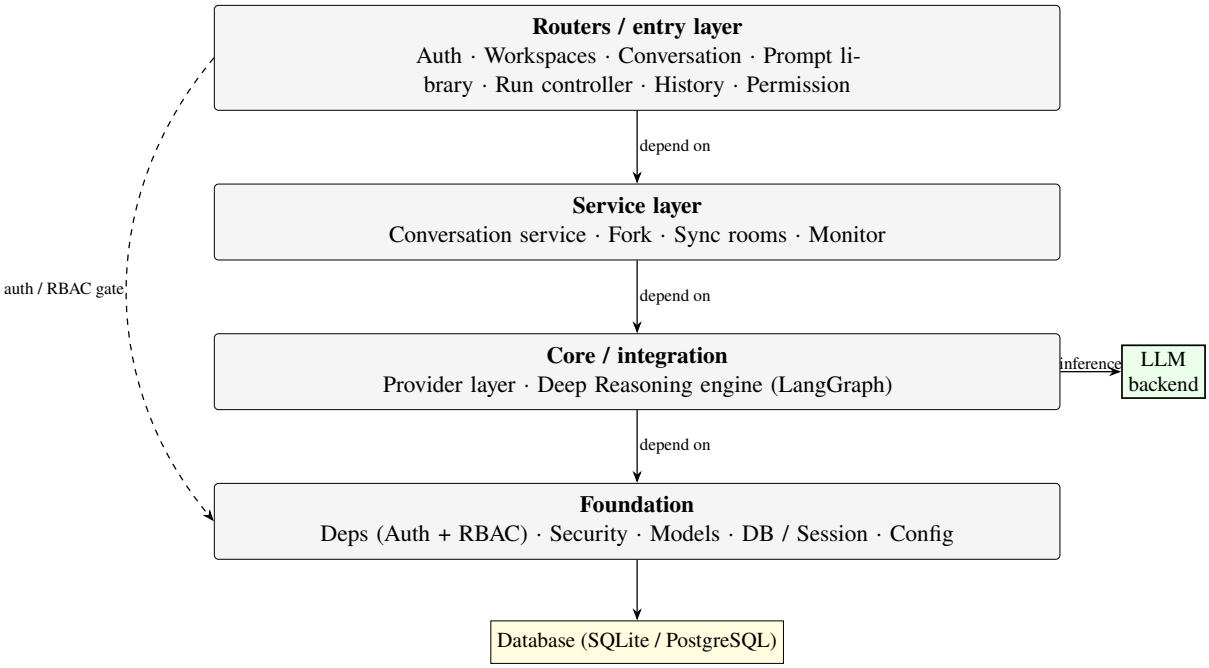

Figure 3.2: Module dependency graph

3.3 Data Flow Diagram

The data-flow diagram is presented at three levels of information. Level 0 (context) shows the team member interacting with the Helix system and the external LLM backend. Level 1 expands the system into its principal processes — authentication, workspace/RBAC, conversation/send, fork, prompt library, deep-reasoning, and monitor — and their data stores. Level 2 details

the Deep Reasoning process: escalate, reason-reflect-synthesize loop, metering, and kill/steer control.

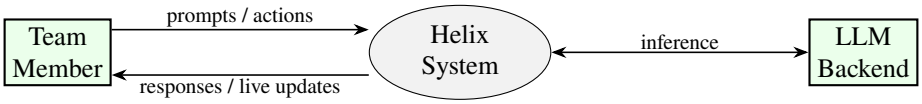

Figure 3.3: DFD Level 0 — context diagram

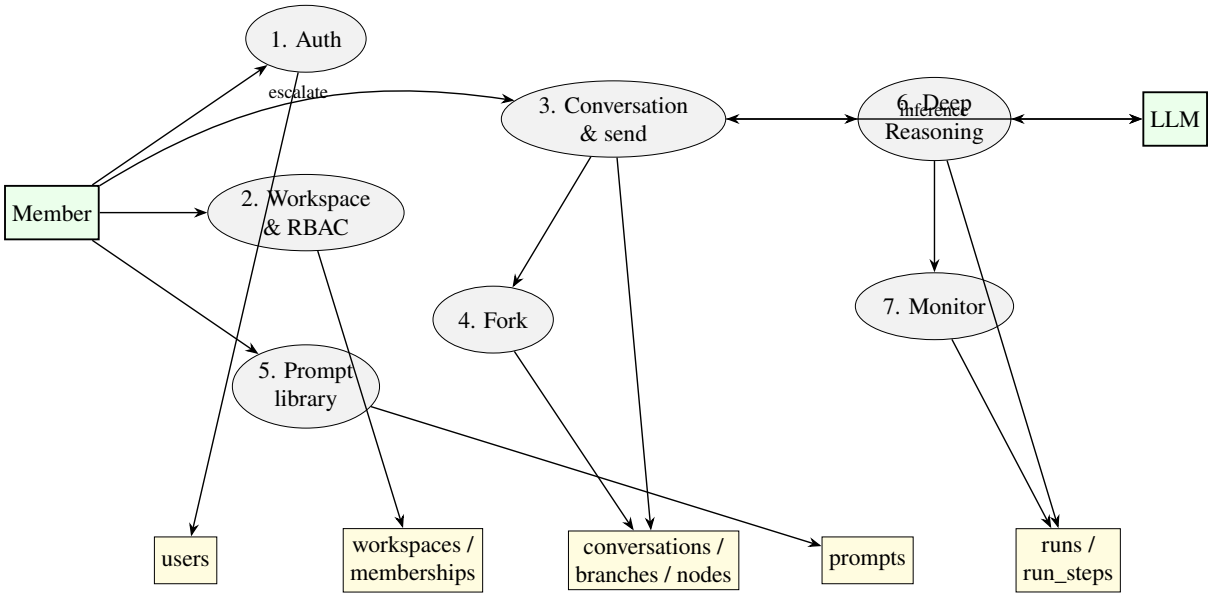

Figure 3.4: DFD Level 1 — principal processes and data stores

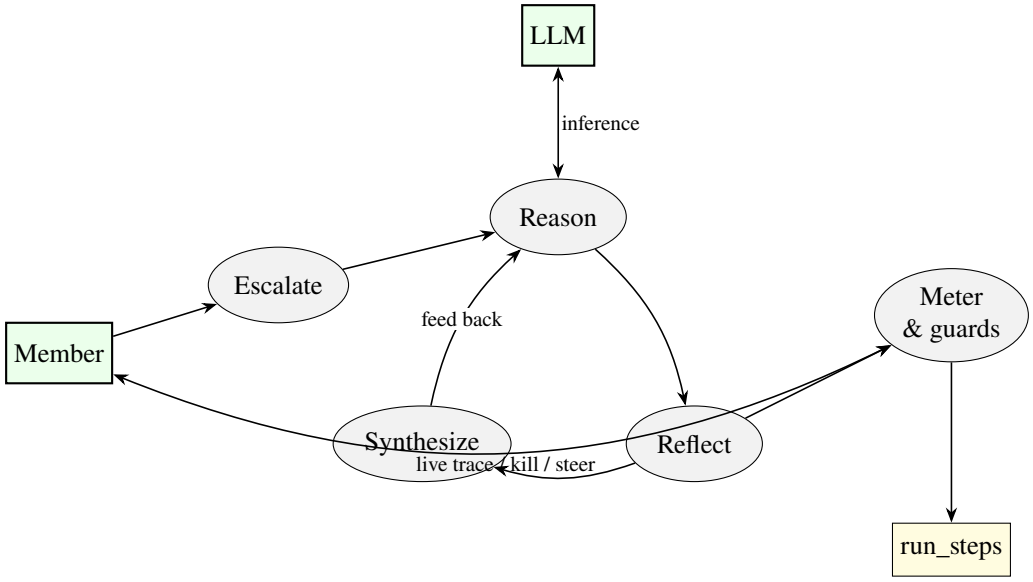

Figure 3.5: DFD Level 2 — Deep Reasoning process

3.4 ER Diagram

The entity-relationship model centres on the workspace as the tenant boundary. Users join workspaces through memberships (many-to-many, carrying the role); a workspace owns conver-

sations, prompts, runs, permissions and invites; a conversation has branches; a branch groups append-only nodes and may be forked from another branch; a run emits ordered run-steps.

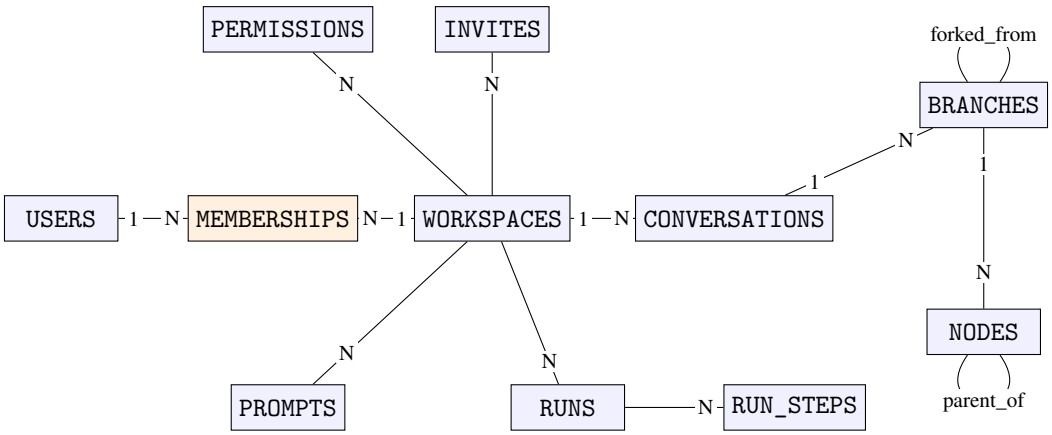

Figure 3.6: Entity-Relationship diagram of the Helix schema

3.5 Database Design

The schema is shaped by five forces: strict multi-tenancy (every team-data row carries a workspace_id), append-only server-ordered conversations (immutable nodes with a monotonic sequence number), structural branching (forks are pointers, not copies), policy-as-data (role permissions live in a table), and polyglot persistence (the same schema runs on SQLite for development and PostgreSQL for production).

3.5.1 Table Design

The principal tables and their structure are given below. Types are portable: on PostgreSQL UUID is native, timestamp is TIMESTAMPTZ and json is JSONB; on SQLite these are stored as TEXT. Enumerations are modelled as VARCHAR + CHECK.

Table: users — the account.

Table 3.3: users table

Column	Type	Constraints	Description
id	UUID	PK	Server-generated identity
email	VARCHAR(255)	UNIQUE, NOT NULL	Login identifier; case-insensitive
pw_hash	VARCHAR(255)	NOT NULL	bcrypt hash of the password
created_at	timestamp	NOT NULL, default now	Registration time

Table: workspaces — the tenant.

Table 3.4: workspaces table

Column	Type	Constraints	Description
id	UUID	PK	Tenant boundary key (on all owned rows)
name	VARCHAR(120)	NOT NULL	Display name
owner_id	UUID	FK → users.id, NOT NULL	The founding Owner
created_at	timestamp	NOT NULL, default now	Creation time

Table: memberships — user-to-workspace association carrying the role.

Table 3.5: memberships table

Column	Type	Constraints	Description
id	UUID	PK	Surrogate identity
user_id	UUID	FK → users.id, NOT NULL	Member
workspace_id	UUID	FK → workspaces.id, NOT NULL	Tenant
role	VARCHAR(16)	NOT NULL, CHECK in {owner, collaborator, observer}	Authorisation role
joined_at	timestamp	NOT NULL, default now	When they joined

Table: conversations — a thread within a workspace, shared or private.

Table 3.6: conversations table

Column	Type	Constraints	Description
id	UUID	PK	Conversation identity
workspace_id	UUID	FK → workspaces.id, NOT NULL	Tenant scope
author_id	UUID	FK → users.id, NOT NULL	Creator (only viewer if private)
title	VARCHAR(200)	NOT NULL	Display title
visibility	VARCHAR(8)	NOT NULL, CHECK in {shared, private}	Access mode
default_branch_id	UUID	FK → branches.id (deferrable)	Root branch
created_at	timestamp	NOT NULL, default now	Creation time

Table: branches — a line in the conversation tree; the table that makes forks O(1).

Table 3.7: branches table

Column	Type	Constraints	Description
id	UUID	PK	Branch identity
conversation_id	UUID	FK → conversations.id, NOT NULL	Owning conversation
name	VARCHAR(120)	NOT NULL	Branch label
parent_branch_id	UUID	FK → branches.id, NULL for root	Lineage link
fork_node_id	UUID	FK → nodes.id, NULL for root	Divergence point
head_node_id	UUID	FK → nodes.id, NULL until first message	Cached tip
engine_thread_id	VARCHAR(64)	NULL	LangGraph checkpoint id (deep-reasoning branches)
created_at	timestamp	NOT NULL, default now	Creation time

Table: nodes — the append-only, immutable message units.

Table 3.8: nodes table

Column	Type	Constraints	Description
id	UUID	PK	Node identity
branch_id	UUID	FK → branches.id, NOT NULL	Branch written on
parent_id	UUID	FK → nodes.id, NULL at root	Previous node (fork spine)
seq	INTEGER	NOT NULL	Monotonic order within branch
author_id	UUID	FK → users.id, NULL for assistant/system	Who sent it
role	VARCHAR(10)	NOT NULL, CHECK in {user, assistant, system}	Message role
content	json	NOT NULL	Message payload
token_count	INTEGER	NOT NULL, default 0	Cached token size
created_at	timestamp	NOT NULL, default now	Write time

Table: prompts — the shared, searchable prompt library.

Table 3.9: prompts table

Column	Type	Constraints	Description
id	UUID	PK	Prompt identity
workspace_id	UUID	FK → workspaces.id, NOT NULL	Tenant scope
author_id	UUID	FK → users.id, NOT NULL	Who saved it
title	VARCHAR(200)	NOT NULL	Short label
body	TEXT	NOT NULL	The prompt text
tags	json	NOT NULL, default []	String tag array
used_count	INTEGER	NOT NULL, default 0	Times inserted (what-works signal)
created_at	timestamp	NOT NULL, default now	Saved at

Table: runs — a Deep-Reasoning escalation of a branch.

Table 3.10: runs table

Column	Type	Constraints	Description
id	UUID	PK	Run identity
workspace_id	UUID	FK → workspaces.id, NOT NULL	Tenant scope
conversation_id	UUID	FK → conversations.id, NOT NULL	Source conversation
branch_id	UUID	FK → branches.id, NOT NULL	Branch reasoned on
status	VARCHAR(10)	CHECK in {running, paused, done, killed, error}	Lifecycle state
provider	VARCHAR(16)	NOT NULL	LLM backend used
model	VARCHAR(64)	NULL	Concrete model name
stop_reason	VARCHAR(32)	NULL	Why it halted
started_at	timestamp	NOT NULL, default now	Start
ended_at	timestamp	NULL	End (null while live)

Table: run_steps — the persisted reasoning trace (one row per engine transition).

Table 3.11: run_steps table

Column	Type	Constraints	Description
id	UUID	PK	Step identity
run_id	UUID	FK → runs.id, NOT NULL	Owning run
idx	INTEGER	NOT NULL	Ordinal within the run
node	VARCHAR(24)	NOT NULL	Engine node (reason/reflect/synthesize)
payload	json	NOT NULL	Step event (thought, energy, depth, readings)
token_count	INTEGER	NOT NULL, default 0	Tokens used this step
latency_ms	INTEGER	NOT NULL, default 0	Step duration
created_at	timestamp	NOT NULL, default now	Emitted at

Table: permissions — RBAC as data (per-workspace, per-role, per-action policy).

Table 3.12: permissions table

Column	Type	Constraints	Description
workspace_id	UUID	PK (composite), FK → workspaces.id	Tenant scope
role	VARCHAR(16)	PK (composite), CHECK in {owner, collaborator, observer}	Role
action	VARCHAR(40)	PK (composite)	Action key (e.g. message.send)
allowed	BOOLEAN	NOT NULL	Whether the role may perform the action

The invites table (token PK, workspace_id, created_by, role, expires_at, created_at) stores opaque, expiring onboarding links and is omitted here for brevity.

3.5.2 Data Integrity and Constraints

- Primary keys are server-generated UUID strings, except invites.token (the secret itself) and the composite PK of permissions.
- Uniqueness: users.email; memberships(user_id, workspace_id); nodes(branch_id, seq); run_steps(run_id, idx).
- Foreign keys use ON DELETE CASCADE down ownership chains (deleting a workspace removes its conversations, branches, nodes, prompts, runs, steps, memberships, permissions and invites) and ON DELETE SET NULL for soft links (nodes.author_id when a user is removed but their messages remain).

- A deferrable constraint links conversations.default_branch_id and branches.conversation_id, satisfied within the create-conversation transaction.
- Not-null discipline: every tenant row's workspace_id is NOT NULL — there is no unscoped resource.
- Visibility is enforced on every read: a conversation is visible where workspace_id = :wid AND (visibility = 'shared' OR author_id = :uid).
- On production PostgreSQL, Row-Level Security policies on every tenant-scoped table constrain access to the active workspace, set per request via a session variable — so even a coding mistake cannot leak across tenants.

3.5.3 Data Dictionary

Domains and enumerations — centralised value sets, each enforced by a CHECK constraint (portable across SQLite and PostgreSQL):

Table 3.13: Domains and enumerations

Domain	Permitted values
role	owner, collaborator, observer (owner > collaborator > observer)
visibility	shared, private
node.role	user, assistant, system
run.status	running, paused, done, killed, error
run.stop_reason	converged, budget, depth_guard, loop_guard, killed, error
provider	stub, groq, ollama

RBAC policy matrix — the permissions table is seeded on workspace creation from the following default matrix (the Owner may later edit it):

Table 3.14: Default RBAC policy matrix

Action	Owner	Collaborator	Observer
conversation.read / replay	Yes	Yes	Yes
message.send	Yes	Yes	No
branch.fork	Yes	Yes	No
prompt.write (save/edit)	Yes	Yes	No
run.escalate (Deep Reasoning)	Yes	Yes	No
run.steer / run.kill	Yes	Yes	No
member.invite / member.role	Yes	No	No

Action	Owner	Collaborator	Observer
permission.edit	Yes	No	No

3.6 Interface and Procedural Design

3.6.1 User Interface Design

The interface follows one governing principle — ornament at the edges, clarity at the centre. The aesthetic ("Alchemical Noir": a near-black canvas, antique-gold line-work, arcane-violet accents) lives in brand moments such as authentication, loaders and the Deep Reasoning monitor, while the working surfaces where members read and write stay calm, legible and high-contrast. The product comprises eight primary screens. Rough wireframes are given below; final screenshots are to be inserted in their reserved spaces.

(1) Authentication (Login / Register)

A ceremonial entry: the brand mark over a slow golden-spiral drift and a single calm card with email and password. A single primary action; errors appear inline beneath the field; the DNA-helix loader replaces the button label while authenticating.

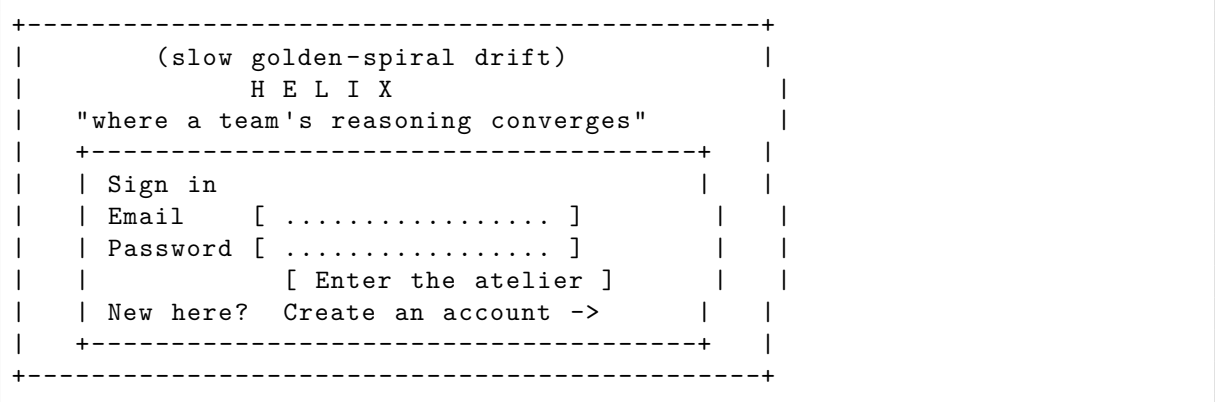

Figure 3.7: UI — Authentication (Login / Register)

(2) Invite Accept

A focused hand-off showing who invited you, into which workspace, and with which role, with one decisive Accept action. The invite preview needs no login; an expired token shows a calm empty state.

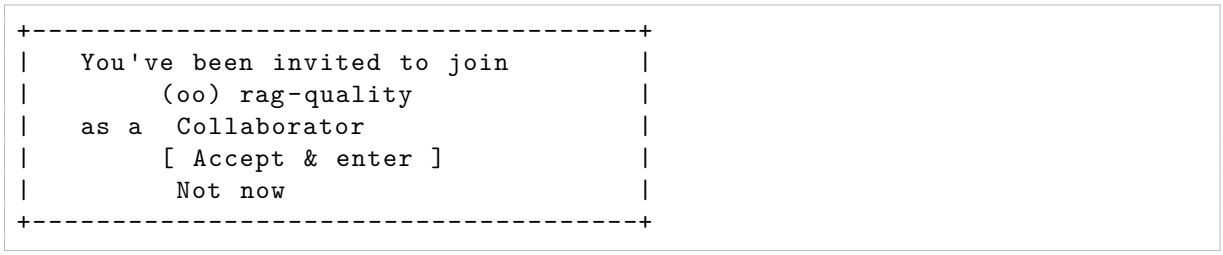

Figure 3.8: UI — Invite Accept

(3) Workspaces - Radial Launcher

A circular navigation wheel: workspace sigils orbit the central mark. Hovering a node shows its name, role glyph, member count and last-active; '+ new' opens a create modal. A plain list is the fallback past about eight workspaces.

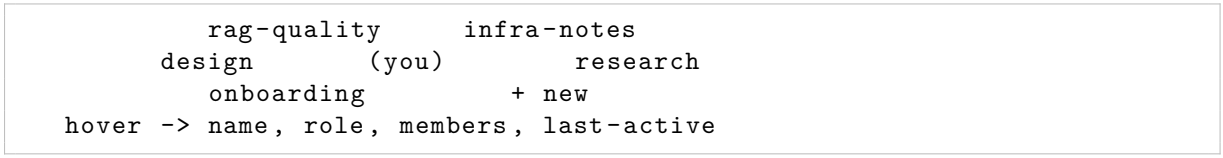

Figure 3.9: UI — Workspaces - Radial Launcher

(4) Workspace Shell - Collaboration View

The everyday surface, split on the golden ratio: a narrow rail (conversations + branch tree), a wide thread, and presence along the top. Visibility sigils mark shared vs private; Fork sits at the thread head; the composer carries Send and a Deep Reason action.

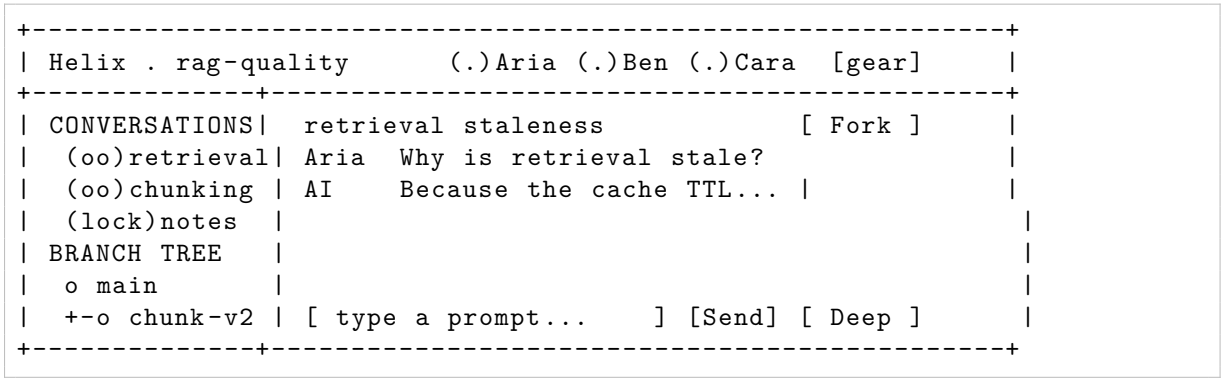

Figure 3.10: UI — Workspace Shell - Collaboration View

(5) Branch Tree

The double-helix made navigable - lineage as a gold spine with helix nodes. Selecting a node opens that branch; 'Fork here' creates a child; the active branch is bright gold; large trees pan and zoom.

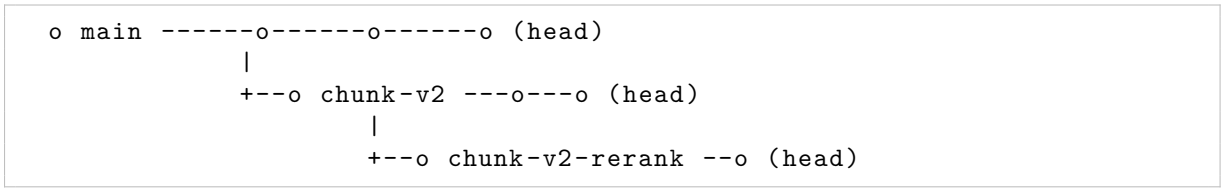

Figure 3.11: UI — Branch Tree

(6) Prompt Library

The team's record of working prompts - calm, searchable and clean. Search filters title/body/tags live; Insert drops the body into the active composer; a use-count is the social proof of what works.

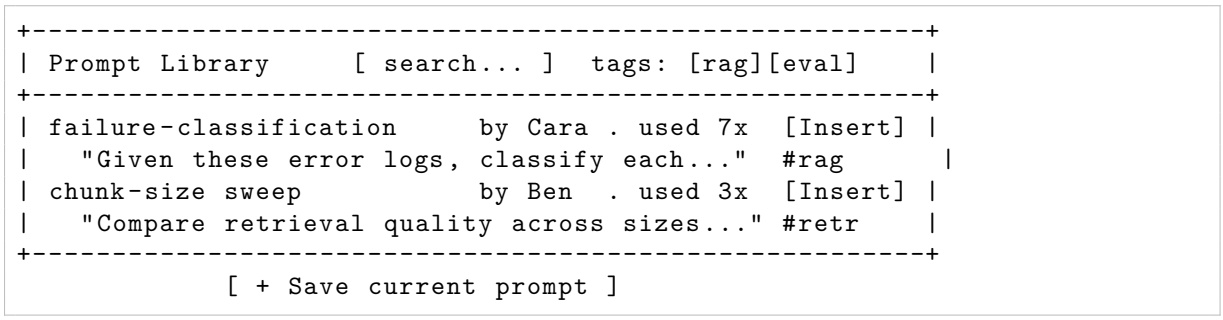

Figure 3.12: UI — Prompt Library

(7) Deep Reasoning Monitor - the Ouroboros

The signature screen: the recursive engine drawn as an ouroboros. Reason, reflect and synthesise sit on a ring traced once per loop; concentric rings show recursion depth; a step trace lists each transition; energy, depth, loop-guard and a budget meter sit below. Kill is an unmissable control; Steer pauses and resumes with guidance. For an Observer, Steer/Kill are absent and the screen reads 'Watching'.

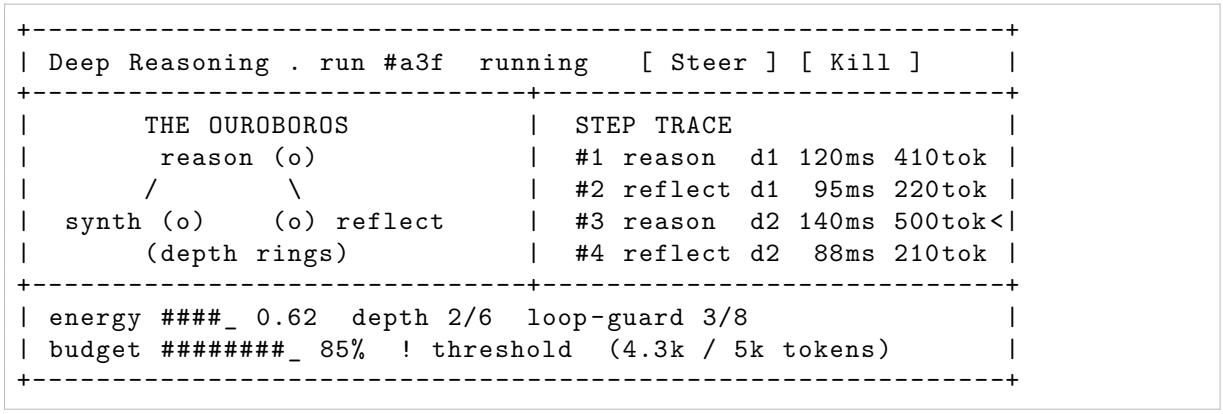

Figure 3.13: UI — Deep Reasoning Monitor - the Ouroboros

(8) Replay & Export

Turning a run into a kept artifact. A scrubber steps through any branch or run node-by-node (the ring re-traces for runs); export offers JSON or Markdown for the current scope.

```
+-------------------------------------------------------+
| Replay . chunk-v2   |<< < ||  > >>    step 6 / 14       |
|  (the thread / ring re-animates to this point)         |
|  Export: [ JSON ]  [ Markdown ]      scope: this       |
+-------------------------------------------------------+
```

Figure 3.14: UI — Replay & Export

Reports / Exports. Helix generates two kept-artifact reports. The Conversation/Run export takes a conversation or run as input (with a scope selector) and outputs an ordered JSON or Markdown document of nodes/steps. The Deep Reasoning run summary outputs the run's status, stop reason, total tokens, depth reached, and the ordered step trace. Reserved space for these report screens is shown below.

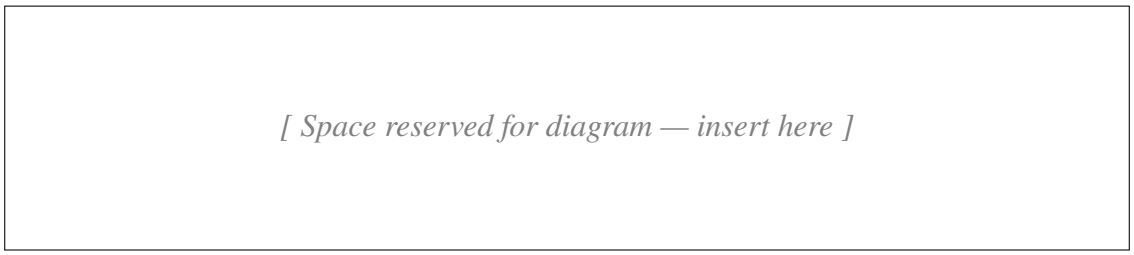

Figure 3.15: Report — Conversation / Run export (JSON / Markdown)

Figure 3.16: Report — Deep Reasoning run summary

3.7 Algorithm Design

This section specifies the non-trivial algorithms at the heart of Helix — each with its purpose, pseudocode, complexity and a note on correctness or termination.

3.7.1 A1 — O(1) Fork and Cross-Branch History Resolution

A fork must be instant regardless of conversation length, so it copies no history; the cost is shifted to a read-time walk up the parent_id spine, which transparently crosses branch boundaries (Git-style structural sharing).

```
FUNCTION fork(cid, N, name, is_deep_reasoning):
    B <- new Branch { conversation_id=cid, name=name,
        parent_branch_id=N.branch_id,   # lineage
        fork_node_id=N.id,              # divergence point
        head_node_id=N.id, engine_thread_id=NULL }
    IF is_deep_reasoning AND branch_of(N).engine_thread_id != NULL:
        B.engine_thread_id <-
   copy_checkpoint(branch_of(N).engine_thread_id)
    persist(B)            # exactly ONE row inserted
    RETURN B

FUNCTION resolve_history(B):
    out <- empty list ; node <- B.head_node   # O(1) via cached head
    WHILE node != NULL:
        out.prepend(node)
        node <- node.parent          # follows parent_id ACROSS branches
    RETURN out
```

Complexity: fork is O(1) writes (one row; plus an O(1) checkpoint-pointer copy for reasoning branches), independent of conversation length; history resolution is O(n) in the number of nodes on the path. Correctness: nodes are immutable and parent_id forms a tree, so a branch merely names a tip and a fork point — no existing node is mutated. The read path, not the write, is where correctness must be tested hardest (crossing the branch boundary at the fork node).

3.7.2 A2 — Two-Stage RBAC Authorization

Authorisation is a fast coarse gate (role rank) followed, where needed, by a fine-grained, data-driven policy lookup. The membership lookup doubles as the tenant-isolation check.

```
ROLE_RANK = { observer:0, collaborator:1, owner:2 }

FUNCTION require_role(user, wid, min_role):      # coarse gate
    m <- membership(user.id, wid)
    IF m = NULL:  RAISE 404 not_found   # non-member: hide tenant
    IF ROLE_RANK[m.role] < ROLE_RANK[min_role]: RAISE 403 forbidden
    RETURN m

FUNCTION can(wid, role, action):                 # policy-as-data
    row <- permissions[wid, role, action]
    IF row != NULL: RETURN row.allowed
    RETURN DEFAULT_MATRIX[role, action]          # default-deny
```

Complexity: O(1) — two point lookups on indexed/PK columns. A non-member fails before any resource is loaded, and the response is 404 (not 403) so the system never discloses that the workspace exists. Any action absent from both the policy and the default matrix resolves to not-allowed (default-deny).

3.7.3 A3 — Recursive Deep Reasoning Loop

The agentic core: reason, reflect, synthesize, feeding each synthesis back as the next thought (the ouroboros), looping until it converges or any one of four independent guards halts it. A step event is emitted per transition for the live monitor and the persisted trace.

```
FUNCTION deep_reason(run, prompt, config):
    state <- { thought:prompt, depth:0, loops:0, tokens:0, energy:1.0 }
    WHILE TRUE:
        IF run.kill_requested:                  RETURN halt('killed')
        IF state.tokens >= config.token_budget: RETURN halt('budget')
        IF state.depth  >= config.max_depth:    RETURN halt('depth_guard')
        IF state.loops  >= config.loop_guard:   RETURN halt('loop_guard')
        IF run.steer_pending: state <- inject(state, await_steer(run))
        reasoning <- LLM.reason(state.thought)          ; emit('reason')
        critique  <- LLM.reflect(reasoning)             ; emit('reflect')
        synthesis <- LLM.synthesize(reasoning,critique);
   emit('synthesize')
        state.tokens += tokens_used(...) ; emit_budget(...)
        state.energy = decay(state.energy)   # strictly decreasing
        IF converged(synthesis) OR state.energy < config.energy_floor:
            RETURN complete(run, synthesis, 'converged')
        state.thought = synthesis ; state.depth += 1 ; state.loops += 1
```

Complexity: O(S) reasoning iterations, bounded above by min(max_depth, loop_guard, token_budget / per_step_tokens). Termination is guaranteed by four independent halts (kill, budget, depth, loop) and a strictly decreasing energy that hits energy_floor — no input can produce an unbounded loop. Because the loop only checks for human input at a step boundary, injected guidance never corrupts mid-transition state.

3.7.4 A4–A7 — Supporting Algorithms

- A4 — Budget metering and threshold alerting. Accumulates per-step tokens, drives the budget meter, raises a one-shot alert at the threshold (an alerted latch prevents spam), and requests a cooperative halt at the next boundary when the budget is reached. O(1) amortised per step.

- A5 — Real-time fan-out with authoritative ordering. A monotonic seq is assigned inside the write transaction (server owns order); broadcast enqueues to per-client bounded queues non-blockingly, so a slow consumer fills only its own queue and is dropped without head-of-line blocking. Clients render by seq, not arrival. O(C) enqueues per event.

- A6 — Prompt library search and ranking. Full-text (GIN) candidate retrieval filtered by tags, ranked by text relevance then used_count (the team's what-works signal) then recency; degrades to a LIKE scan on SQLite. Sub-linear retrieval + O(k log k) ranking.

- A7 — Invite token lifecycle. A 256-bit url-safe random token is the primary key; acceptance is a PK lookup with UTC-normalised expiry checking, idempotent re-accept, and

not-found for expired tokens to avoid leaking workspace existence. O(1).

Table 3.15: Algorithm complexity summary

#	Algorithm	Time	Space	Key guarantee
A1	Fork (write)	O(1)	O(1)	No history copied
A1	History resolution (read)	O(n)	O(n)	Correct cross-branch walk
A2	RBAC authorization	O(1)	O(1)	Default-deny; tenant-isolating
A3	Deep Reasoning loop	O(S)	O(1) state	Guaranteed termination (4 guards + energy)
A4	Budget metering	O(1)/step	O(k) window	One-shot alert; cooperative halt
A5	Fan-out + ordering	O(C)/event	O(C) queues	Total order by seq; no HOL blocking
A6	Library search	sub-linear + O(k log k)	O(k)	Relevance then proven usefulness
A7	Invite lifecycle	O(1)	O(1)	Unguessable, expiring, idempotent